

RÉPUBLIQUE DU CAMEROUN

REPUBLIC OF CAMEROON

Peace - Work - Fatherland

UNIVERSITÉ DE DSCHANG

UNIVERSITY OF DSCHANG

Scholae Thesaurus DschangensisIbiCordum

BP 96, Dschang (Cameroun) - Tél./Fax (237) 233 45 13 81 Website : <http://www.univ-dschang.org>.

E-mail : udsrectorat@univ-dschang.org



FACULTE DES SCIENCES

FACULTY OF SCIENCE

**Département de
Mathématiques-Informatique**

*Department of Mathematics and
Computer Science*

BP 67, Dschang (Cameroun) - Tél./Fax (237) 243 69 15 10 E-mail : dept.mi@univ-dschang.org

PROJET DE SYSTÈME D'EXPLOITATION ET RÉSEAUX

THEME : INSTALLATION ET CONFIGURATION D'UN SERVEUR TFTP

Présenté par :

NOMS ET PRENOMS

MATRICULES

Option: Informatique Fondamentale

Niveau: Licence 3

Sous la coordination de :

Pr KENGNE T. Vianney

Contents

INTRODUCTION	3
I- PRESENTATION DU SERVEUR FTP	4
I-1) HISTORIQUE	4
I-2) FONCTIONNEMENT	4
I-2-a) Mode de fonctionnement	4
I-2-b) LES PAQUETS ET COMMANDES TFTP	5
I-3-c) CLIENTS TFTP ET SERVEURS TFTP	7
I-3) INSTALLATION ET CONFIGURATION D'UN SERVEUR TFTP (SOUS WINDOWS)	10
I-3-a) INSTALLATION.....	10
I-3-b) CONFIGURATION	11
I-3-c) Test Du serveur TFTP	13
I-4) SECURITES ET VULNERABILITES	14

INTRODUCTION

Dans cet article, nous allons découvrir trois protocoles incontournables : TFTP, FTPS et SFTP. Ces trois protocoles servent à effectuer du transfert de fichiers, aussi bien pour envoyer que recevoir, mais avec des différences notamment au niveau de la sécurité.

Le TFTP (Trivial File Transfer Protocol) est un protocole de transfert de fichiers simple mais non sécurisé, le SFTP (SSH File Transfer Protocol) repose sur le protocole SSH pour des transferts sécurisés chiffrés, tandis que le FTPS (FTP Secure) utilise SSL/TLS pour ajouter une couche de sécurité aux transmissions FTP en chiffrant les données et en offrant des certificats SSL, ce qui en fait une option sécurisée pour les échanges sensibles. Chacun de ces protocoles présente ses propres caractéristiques en matière de sécurité et de confidentialité, avec le FTPS et le SFTP offrant des solutions robustes pour garantir la protection des données lors des transferts de fichiers.

I- PRESENTATION DU SERVEUR FTP

I-1) HISTORIQUE

A ses origines, le protocole TFTP a été défini pour la première fois en 1980 par l'IEN 133. Karen R. Sollins a été créditée comme étant la développeuse de TFTP. Sa standardisation initiale via RFC 783 a eu lieu en Juin 1981 ; ledit protocole a été influencé par le protocole antérieur EFTP, qui faisait partie de la suite des protocoles PARC Universal Packet.

I-2) FONCTIONNEMENT

I-2-a) Mode de fonctionnement

TFTP (Trivial File Transfer Protocol) est un protocole simplifié de transfert de fichiers qui fonctionne selon un modèle **client-serveur**, à l'instar de FTP. Toutefois, contrairement à FTP, il est bien plus léger, ne nécessite **aucune authentification** et fonctionne uniquement via le protocole **UDP** (User Datagram Protocol).

TFTP utilise **un seul canal de communication** basé sur UDP pour l'ensemble de ses échanges, notamment :

- L'envoi de requêtes de lecture (RRQ – Read Request)
- L'envoi de requêtes d'écriture (WRQ – Write Request)
- La transmission de blocs de données (DATA)
- Les accusés de réception (ACK)
- Les messages d'erreur (ERROR)

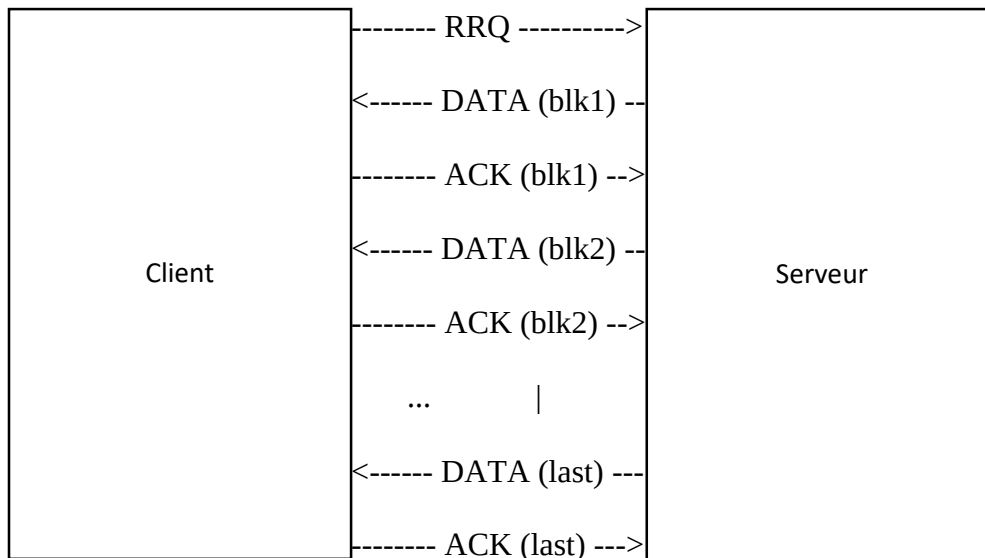
Le protocole TFTP repose sur le port **69/UDP** pour initier les requêtes. Après la requête initiale, le serveur et le client communiquent via **des ports UDP dynamiques**, choisis aléatoirement, ce qui permet de gérer plusieurs transferts simultanés.

En ce qui concerne le fonctionnement général du protocole TFTP, il est comme suit :

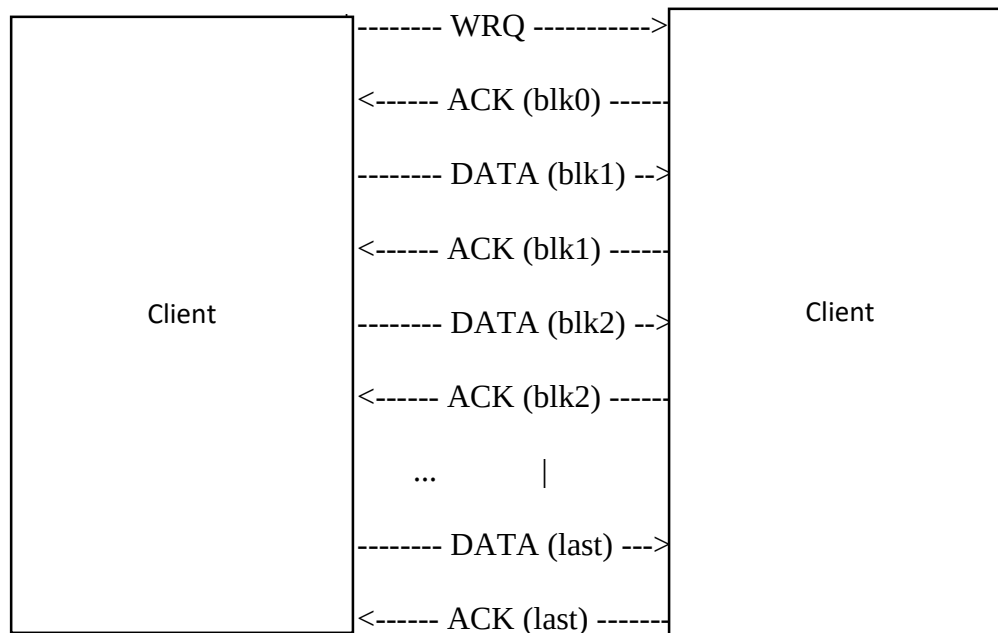
- Le **client TFTP** envoie une requête RRQ (lecture) ou WRQ (écriture) au **serveur TFTP** sur le port **69/UDP**.
- Le **serveur TFTP** répond en utilisant un **port UDP temporaire**, différent de 69, pour toute la suite de la communication.
- Le fichier est ensuite découpé en **blocs de 512 octets** maximum, envoyés un par un.

- Chaque bloc envoyé doit être **confirmé par un ACK** (accusé de réception) du client.
- Le transfert s'arrête lorsque :
 - un bloc est **inférieur à 512 octets** (ce qui indique la fin du fichier),
 - ou qu'un **message d'erreur** est envoyé (problème de permissions, fichier introuvable, etc.).

Exemple schématique du processus de transfert (cas de la lecture d'un fichier par le client):



Exemple schématique du processus de transfert (cas de la lecture d'un fichier par le client):



I-2-b) LES PAQUETS ET COMMANDES TFTP

Le protocole TFTP est très simple et ne possède qu'un nombre **très limité de commandes**, comparé à par exemple FTP. Il fonctionne selon un échange de **messages standardisés** définis dans la **RFC 1350**, chacun étant représenté par un **code opération (Opcode)**.

Voici les **5 commandes principales de TFTP** :

Opcode	Commande	Nom	Description
1	RRQ	Read Request	Requête de lecture d'un fichier à partir du serveur TFTP.
2	WRQ	Write Request	Requête d'écriture d'un fichier vers le serveur TFTP.
3	DATA	Data Packet	Bloc de données transmis, maximum 512 octets.
4	ACK	Acknowledgment	Accusé de réception d'un bloc DATA reçu.
5	ERROR	Error Packet	Message d'erreur retourné si une opération échoue.

Ces paquets sont en quelque sorte des messages generees par le client TFTP ou le serveur TFTP en fonction des commandes executees. Ces differentes commandes sont referencees dans le tableau suivant :

Commande	Fonction
<code>get <fichier></code>	Télécharger un fichier du serveur vers le client
<code>put <fichier></code>	Envoyer un fichier du client vers le serveur
<code>mode <mode></code>	Définir le mode de transfert (par défaut : octet)
<code>connect <IP></code>	Se connecter à un serveur TFTP
<code>status</code>	Affiche l'état actuel du client TFTP
<code>verbose</code>	Active/désactive les messages détaillés
<code>trace</code>	Affiche les paquets échangés (utile pour debug)
<code>quit</code>	Quitte le client TFTP
<code>? ou help</code>	Affiche l'aide avec les commandes disponibles

Lors d'un transfert (avec get ou put), tu peux choisir un **mode**, qui détermine comment les données sont traitées :

Mode	Description
<code>octet</code>	Transfert binaire brut (images, exécutables, PDF, etc.) → mode le plus utilisé
<code>netascii</code>	Transfert texte ASCII normalisé (utile pour les fichiers texte multi-OS)
<code>mail</code>	(Obsolète, rarement utilisé) — pour envoyer un fichier à une boîte mail

I-3-c) CLIENTS TFTP ET SERVEURS TFTP

Les **clients TFTP** et les **serveurs TFTP** sont des composants essentiels pour le transfert de fichiers via le protocole **TFTP (Trivial File Transfer Protocol)**. Bien que TFTP soit plus simple et moins sécurisé que le FTP, il est largement utilisé pour les transferts dans des environnements où la simplicité et la rapidité sont les priorités, comme dans les réseaux embarqués ou pour la mise à jour de firmware.

Voici un aperçu détaillé de chacun :

Presentation des serveurs et des clients TFTP

a-)serveurs TFTP

Un **serveur TFTP** est une application logicielle qui s'exécute sur un ordinateur ou un périphérique réseau distant. Il permet aux utilisateurs ou aux équipements connectés d'accéder à ses fichiers via le protocole TFTP. Le serveur héberge les fichiers et les répertoires accessibles à des clients TFTP, avec des règles de sécurité limitées, car TFTP ne propose pas de mécanismes d'authentification ou de chiffrement.

Serveurs TFTP sur Windows

Serveur TFTP	Avantages	Inconvénients
TFTPD32 / TFTPD64	- Simple à utiliser, interface graphique claire. - Permet de gérer à la fois les clients et les serveurs TFTP. - Léger et rapide.	- Pas d'authentification, donc peu sécurisé. - Peut ne pas être aussi robuste dans des environnements très exigeants. - Moins performant sous des charges élevées.
SolarWinds TFTP Server	- Fiable et utilisé dans des environnements professionnels. - Interface facile à configurer.	- Version gratuite avec fonctionnalités limitées. - Plus adapté aux entreprises que pour un usage personnel.
OpenTFTPServer	- Open-source et gratuit. - Peut gérer de nombreux transferts simultanés.	- Moins connu, donc moins de support communautaire. - Interface graphique moins intuitive.

Serveurs TFTP sur LINUX

Serveur TFTP	Avantages	Inconvénients
tftpd-hpa	- Très léger et rapide. - Très stable et robuste, utilisé dans les systèmes embarqués.	- Pas d'interface graphique (doit être configuré via ligne de commande). - Moins accessible pour les utilisateurs non expérimentés.
atftpd	- Très répandu dans les distributions Linux. - Supporte plusieurs threads, idéal pour des transferts parallèles. - Compatible avec des systèmes à grande échelle. - Open-source, hautement configurable.	- Manque de sécurité (aucune authentification). - Configuration plus complexe comparé à d'autres serveurs TFTP. - Moins populaire que <code>tftpd-hpa</code>, donc moins de documentation. - Moins convivial pour les débutants.

b-) Clients TFTP

Un **client TFTP** est une application qui permet à un utilisateur de se connecter à un **serveur TFTP** pour transférer des fichiers. Les clients TFTP sont souvent utilisés dans des scénarios où les fichiers doivent être transférés de manière rapide et simple, souvent sans authentification. Ces clients offrent généralement une interface de ligne de commande (CLI) pour faciliter les transferts.

Clients TFTP sous Windows

Client TFTP	Avantages	Inconvénients
TeraTerm	- Interface graphique avec support TFTP via scripts. - Très fiable pour des transferts simples.	- Principalement utilisé pour des connexions série, moins dédié à TFTP. - L'interface de transfert TFTP est moins flexible.
PumpKIN	- Interface graphique simple et claire. - Supporte à la fois le rôle client et serveur.	- Fonctionnalité basique, pas d'options avancées. - Pas aussi puissant ou flexible que d'autres clients.
WinSCP	- Très populaire pour les transferts SFTP et FTP, mais peut aussi être utilisé pour TFTP. - Interface graphique très soignée.	- Pas aussi optimisé pour TFTP. - Moins utilisé pour des transferts TFTP spécifiques.

Clients TFTP sous LINUX

Client TFTP	Avantages	Inconvénients
TeraTerm	- Interface graphique avec support TFTP via scripts. - Très fiable pour des transferts simples.	- Principalement utilisé pour des connexions série, moins dédié à TFTP. - L'interface de transfert TFTP est moins flexible.
PumpKIN	- Interface graphique simple et claire. - Supporte à la fois le rôle client et serveur.	- Fonctionnalité basique, pas d'options avancées. - Pas aussi puissant ou flexible que d'autres clients.
WinSCP	- Très populaire pour les transferts SFTP et FTP, mais peut aussi être utilisé pour TFTP. - Interface graphique très soignée.	- Pas aussi optimisé pour TFTP. - Moins utilisé pour des transferts TFTP spécifiques.

I-3) INSTALLATION ET CONFIGURATION D'UN SERVEUR TFTP (SOUS WINDOWS)

I-3-a) INSTALLATION

Serveur TFTP : TFTPd32 / TFTPd64

TFTPd32 (ou TFTPd64 pour les systèmes 64 bits) est un logiciel open-source pour Windows qui offre plusieurs services réseau :

- **Serveur TFTP**
- Serveur DHCP
- Serveur Syslog
- Serveur SNTP
- Client TFTP

Il est léger, rapide et facile à utiliser, souvent utilisé pour les mises à jour de firmware de routeurs, commutateurs, caméras IP, etc.

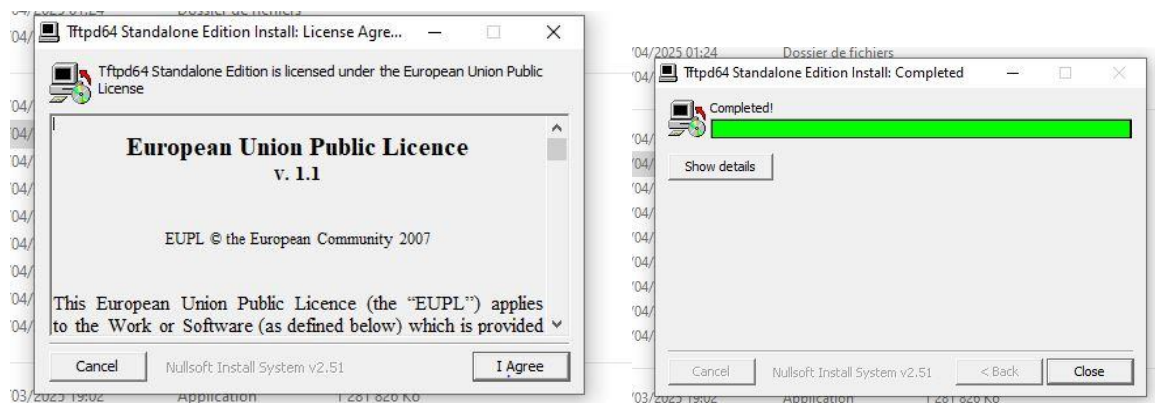
Étapes d'installation :

1. Télécharger le logiciel

- Va sur le site officiel :
<http://tftpd32.jounin.net>
- Télécharge la version qui correspond à ton système (32 ou 64 bits).

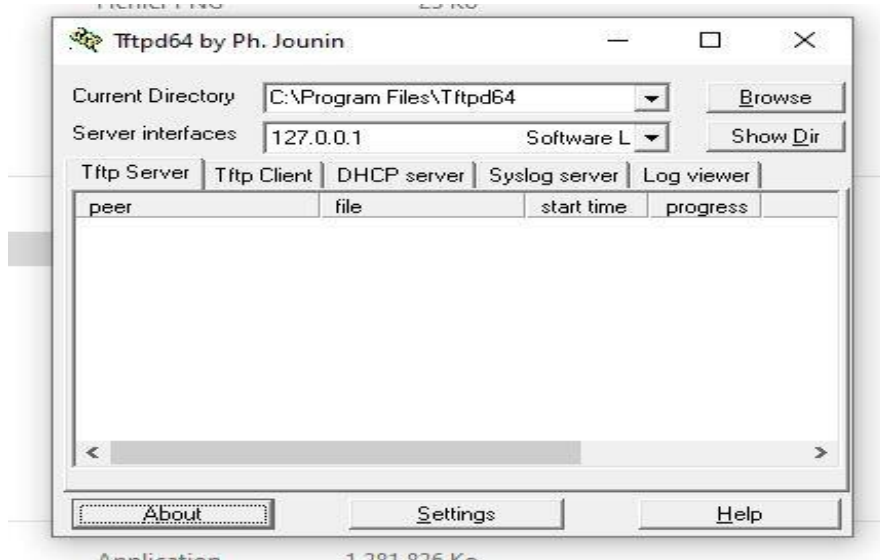
2. Lancer l'installation

- Double-clique sur l'exécutable téléchargé.
- Suis les étapes de l'assistant d'installation.



3. Lancer TFTP32/TFTP64

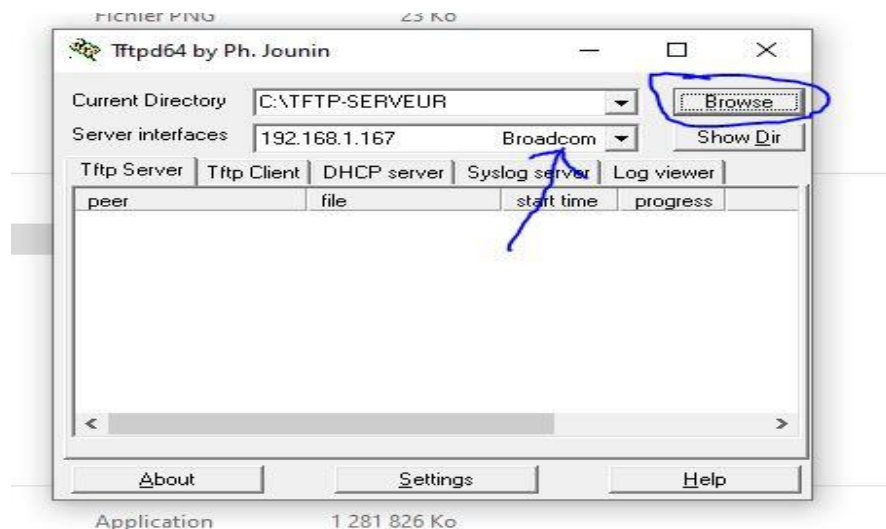
- Cliquez droit sur le raccourci "**Exécuter en tant qu'administrateur**".
- L'interface s'ouvre, avec différents onglets (TFTP, DHCP, Syslog...).



I-3-b) CONFIGURATION

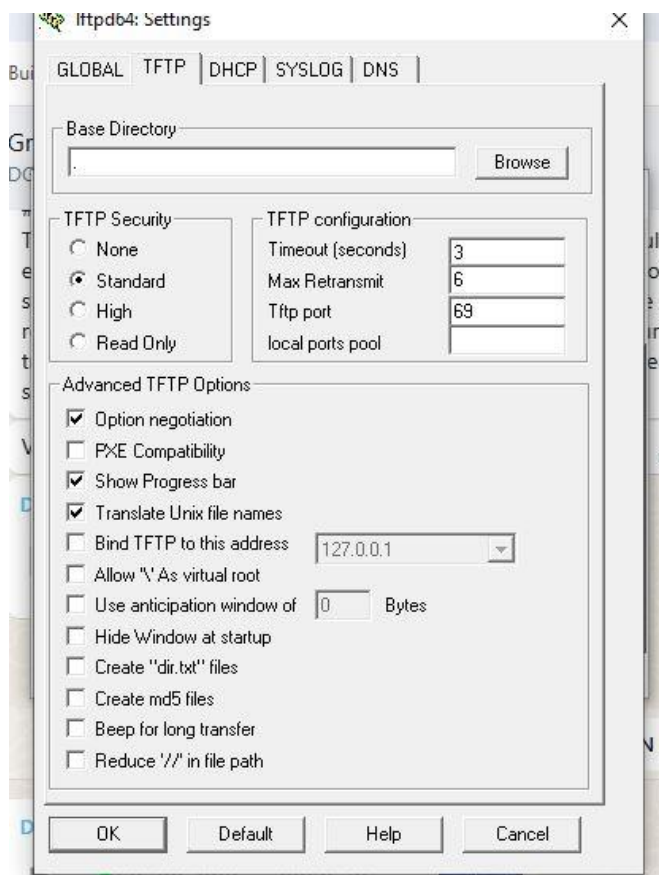
(SUR WINDOWS)

1. **Ouvrir l'onglet "TFTP"** dans le logiciel.
2. **Configurer le "Current Directory"**
 - C'est le dossier local à partir duquel les fichiers seront envoyés ou reçus.
 - Exemple : C:\TFTP-SERVEUR
 - Crée manuellement ce dossier s'il n'existe pas.
 - Dans la fenêtre principale, cherche le champ "**Server interface**" (ou "Serveur TFTP"). Il s'agit d'un menu déroulant listant **toutes les interfaces réseau disponibles**, avec leurs **adresses IP** associées.
 - Sélectionne l'**adresse IP correspondant au réseau local** utilisé (exemple : 192.168.1.100).



3. Cocher les bonnes options :

- TFTP Server (activé par défaut)
- Create « dir.txt » files
- PXE Compatibility
- Show Progress bar



4. (Optionnel) Autoriser le port 69 dans le **pare-feu Windows** :

- Ouvre le **Panneau de configuration > Pare-feu Windows Defender**
- Autorise **TFTPD32.exe** ou **TFTPD64.exe**
- Assure-toi que le **port UDP 69** est ouvert.

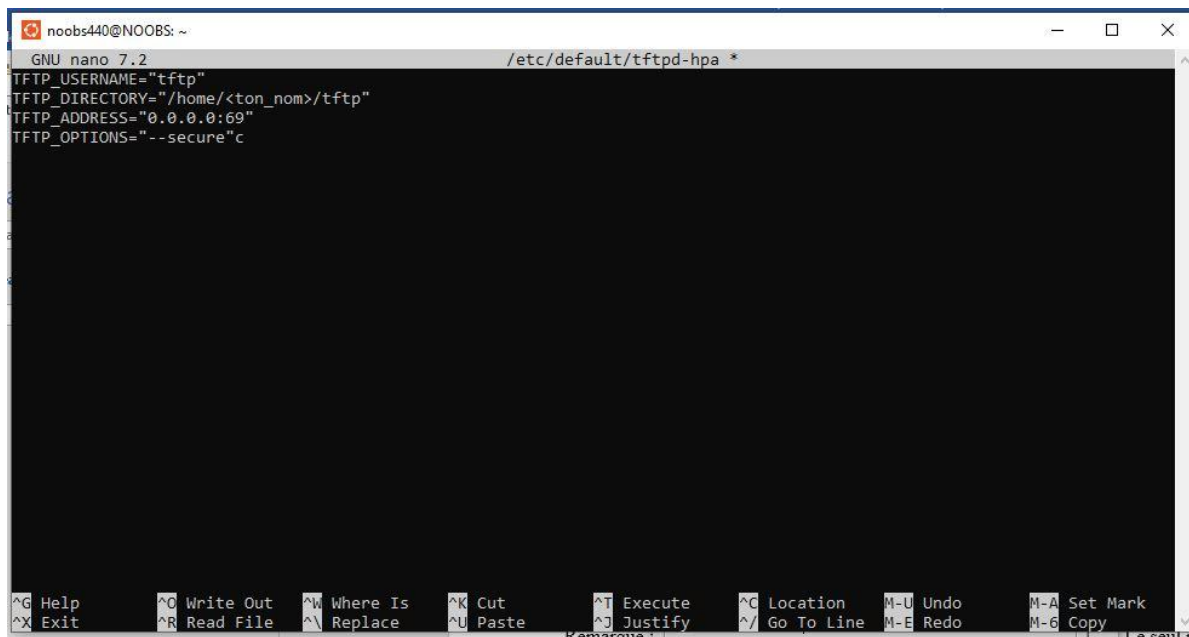
(SUR LINUX)

1. Installer les paquets nécessaires

```
noobs440@NOOBS:~$ sudo apt install tftpd-hpa
```

2. Configurer les options du serveur

```
noobs440@NOOBS:~$ sudo nano /etc/default/tftpd-hpa
```



The screenshot shows a terminal window with the nano text editor open. The file being edited is /etc/default/tftpd-hpa. The content of the file is as follows:

```
GNU nano 7.2 /etc/default/tftpd-hpa *
TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/home/<ton_nom>/tftp"
TFTP_ADDRESS="0.0.0.0:69"
TFTP_OPTIONS="--secure"c
```

The bottom of the window shows the nano editor's command palette with various shortcuts like ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, M-U Undo, M-A Set Mark, ^X Exit, ^R Read File, ^W Replace, ^U Paste, ^J Justify, ^_ Go To Line, M-E Redo, and M-G Copy.

3. Surtout ne pas oublier d'ouvrir le port UDP 69

Remarque : Le cote client ne necessite a peu pres aucune configuration. Le seul prerequis est d'etre connecte au meme reseau local que l'hote serveur.

I-3-c) Test Du serveur TFTP

Depuis une machine cliente (Linux ou Windows) tu peux tester un envoi ou une réception avec une commande comme :

```
noobs440@NOOBS:~$ tftp 192.168.1.1
tftp> put monfichier.txt
```

Sur LINUX

Sur Windows

```
C:\Users\LENOVO>tftp 192.168.1.1 put monfichier.txt
```

I-4) SECURITES ET VULNERABILITES

a) L'absence d'authentification

TFTP ne demande ni nom d'utilisateur ni mot de passe. Tout utilisateur ayant accès au réseau peut lire ou écrire des fichiers sur le serveur TFTP.

b) Données transmises en clair

Les fichiers transitent en texte brut (pas de chiffrement). Un attaquant en écoute (sniffing) peut capturer des données sensibles :

- Configurations réseau.
- Mots de passe en clair.

c) Faiblesse du protocole UDP

Lors d'une attaque par exemple DoS l'absence de gestion de congestion UDP facilite les attaques par injection massive de paquets.

II- PRESENTATION DU SERVEUR FTPS

II-1) HISTORIQUE

Le protocole FTPS est apparu à la fin des années 1990, en réponse aux **failles de sécurité du FTP classique**. Il s'appuie sur le **protocole SSL (Secure Sockets Layer)** puis **TLS (Transport Layer Security)**, utilisés également pour sécuriser le web (HTTPS). FTPS a été conçu pour permettre la **continuité du FTP classique**, tout en y ajoutant une couche de chiffrement.

II-2) FONCTIONNEMENT

II-2-a) Mode de fonctionnement

FTPS est une version du FTP qui intègre une **couche de chiffrement SSL/TLS** pour sécuriser les connexions entre le client et le serveur. Il s'agit donc d'un **FTP sécurisé**, avec compatibilité des commandes FTP classiques.

Son mode de fonctionnement est décrit tel que suit :

1. Ouverture de connexion (port 21 ou 990)

- En **mode explicite** : le client se connecte sur le port 21 et demande explicitement l'activation du chiffrement via la commande `AUTH TLS`.
- En **mode implicite** : la connexion commence déjà chiffrée sur le port 990.

2. Échange de certificats

- Le serveur envoie un **certificat SSL/TLS** pour prouver son identité.
- Le client peut vérifier ce certificat (optionnellement avec une autorité de certification).

3. Négociation SSL/TLS

Les deux parties établissent un **canal de communication chiffré**.

4. Authentification de l'utilisateur

Le client s'identifie avec un **nom d'utilisateur et un mot de passe**, ou avec un **certificat client** si nécessaire.

5. Navigation / transfert de fichiers

- Le client envoie des commandes (`LIST`, `RETR`, `STOR`...) via le canal de contrôle chiffré.

- Les **données** sont transmises sur un canal **séparé**, qui peut aussi être chiffré.

6. Fermeture de la session

II-2-b) Avantages et Inconvénients

Avantages

- **Sécurisé** : Chiffre les données, donc elles sont protégées pendant le transfert.
- **Authentification** : Utilise des certificats pour vérifier l'identité des utilisateurs et des serveurs.
- Communément utilisé : Bien connu et supporté par de nombreux outils de transfert de fichiers.

Inconvénients

- **Complexe à configurer** : Mettre en place peut être difficile, surtout avec les certificats.
- **Problèmes avec les pare-feu** : Peut nécessiter plusieurs ports, ce qui peut bloquer les connexions.
- **Pas idéal pour les connexions instables** : Peut rencontrer des problèmes de performance sur des réseaux peu fiables.

III- PRESENTATION SU SERVEUR SFTP

III-1)HISTORIQUE

Le protocole SFTP a été développé par l'**IETF (Internet Engineering Task Force)** dans le cadre du projet SSH2. Il est né au début des années 2000, comme une **solution de remplacement sécurisée à FTP**, entièrement basée sur le protocole **SSH (Secure Shell)**. Contrairement à FTPS, SFTP ne dépend **pas du tout du FTP**.

III-2)FONCTIONNEMENT

III-2-a)Mode de Fonctionnement

SFTP fonctionne **au-dessus du protocole SSH** (Secure Shell). Il est totalement différent de FTP/FTPS : il n'y a **qu'un seul canal chiffré**, qui gère à la fois les **commandes et les données**.

Mode de fonctionnement :

1. Connexion SSH (port 22)

Le client établit une connexion SSH sécurisée avec le serveur.

2. Échange de clés (SSH handshake)

- Le serveur envoie sa **clé publique SSH**.
- Le client l'utilise pour établir une **connexion sécurisée et authentifiée**.

3. Authentification de l'utilisateur

Par **mot de passe** ou **clé privée** (souvent utilisée pour les connexions automatiques).

4. Ouverture d'une session SFTP

Une fois connecté via SSH, le client lance la **sous-session SFTP**.

5. Échange de commandes et données

- Toutes les opérations (transfert, suppression, renommage de fichiers, etc.) passent dans le **même tunnel sécurisé**.
- Pas besoin de ports supplémentaires.

6. Fermeture de la session

Le tunnel SSH se ferme et la session SFTP est terminée.

III-2-b) Avantages et inconvénients

Avantages

- **Très sécurisé** : Chiffre toutes les données, donc tout est protégé.
- **Facile à utiliser** : Utilise un seul port, ce qui simplifie la configuration.
- **Authentification forte** : Vérifie les utilisateurs avec des clés, ce qui est très sécurisé.

Inconvénients

- **Fonctionnalités limitées** : Moins de fonctionnalités avancées par rapport à FTP traditionnel.
- **Dépendant de SSH** : Nécessite que SSH soit installé et configuré.
- **Peut être lent sur des réseaux lents** : Le chiffrement peut ralentir les transferts sur des connexions à haute latence.